VynFi-Net: A Reproducible Synthetic Benchmark for Accounting-Network Anomaly Detection

Michael Ivertowski
Independent Researcher
Zurich, Switzerland
https://orcid.org/0009-0008-7829-2249

May 2026

Abstract

Machine-learning research on accounting-anomaly detection has produced a steady stream of methodological advances over the past decade—deep autoencoders for journal-entry outliers [28], adversarial detectors for accounting deepfakes [29], graph neural networks for transaction-graph fraud, and a wide variety of tabular baselines. Yet the field has no shared, reproducible benchmark. Every published result is reported on a private dataset—typically a single client's general ledger held under audit-confidentiality constraints—which makes cross-paper comparison impossible and slows the accumulation of knowledge that public benchmarks [5, 9, 19] have driven in neighbouring fields.

We close this gap with VynFi-Net, an open, fully-provenanced reference benchmark for accounting-network anomaly detection. VynFi-Net is built on DataSynth [16], a Rust framework that generates end-to-end coherent synthetic enterprise datasets where every node, edge, and property derives from a known generative process. The empirical foundation of the benchmark is an internal study of 364 million journal-entry postings drawn from 155 real-world general ledgers [14], from which we recover the line-item distribution and structural network properties that DataSynth implements.

The benchmark contributes four reusable artefacts. *First*, a 13-column flat edge schema (`graphs/je_network.csv`) that materialises five mathematically-equivalent network-construction methods (Methods A–E) over the journal-entry table, with deterministic UUID v5 edge identifiers and a `predecessor_line_id` field that traces booking chains across documents. *Second*, a labelled anomaly-injection layer covering more than 130 subtypes—spanning fraud, error, process-control, statistical, and relational categories—with every flagged edge linked back to a ground-truth event class, enabling both binary and multi-class tasks without re-labelling. *Third*, three reproducible evaluation splits—temporal (last three fiscal months held out), entity (ten percent of subsidiaries held out), and fraud-typology (typology classes held out)—together with a leakage audit so that any future submission can be checked against the same data partition. *Fourth*, a baseline suite spanning graph neural networks (GCN, GraphSAGE, GAT, GIN, RGCN), tabular gradient boosting (XGBoost), and classical anomaly detectors (Isolation Forest, Local Outlier Factor), together with a simple rule-based reference, all trained from a single declarative configuration.

We report initial numbers on the released configuration. Strong GNN baselines reach competitive area-under-precision-recall on the binary task, but cross-typology generalisation

under the typology-hold-out split remains substantially harder—suggesting that structural representations alone do not fully transfer across fraud schemes. The full benchmark—data-generation configuration, generated dataset, edge-list export, splits, baselines, and weights—is released under a permissive licence at the DataSynth repository [15].

VynFi-Net does not replace empirical work on real client data. Its role is complementary: a reference frame against which methods can be trained, ablated, and compared without the confidentiality concerns that block public release of the underlying ledgers.

Keywords: accounting networks, journal entries, synthetic benchmarks, graph neural networks, anomaly detection, fraud detection, audit analytics, reproducibility, ISO 21378, reference knowledge graphs

1 Introduction

1.1 The benchmark gap in machine-learning for audit

Machine-learning approaches to accounting and audit analytics have matured rapidly. Deep autoencoder networks were shown to surface journal-entry outliers at scale [28]; adversarial training was extended to detect synthetic "accounting-deepfake" postings [29]; gradient-boosted trees, density-based outlier detectors, and—more recently—graph neural networks have all been reported as effective on particular fraud and error schemes [8, 34]. In parallel, the auditing standards themselves have moved towards explicit endorsement of analytical procedures [12, 32] and a standardised data-exchange schema [13], both of which presuppose that practitioners can compare detection methods on comparable data.

Yet the empirical literature on accounting-anomaly detection has, to a remarkable degree, advanced *without a shared benchmark*. Almost every published evaluation operates on a private dataset—typically a single client's general ledger, held under audit-confidentiality constraints—and reports figures that are reproducible only by the authors and by readers with access to the same proprietary system. Two effects follow. First, methodological comparison across papers is not possible: a graph attention network reported as "state of the art" on one corporate ledger may underperform a simple rule-based detector on another, and the field has no way of detecting this from the published numbers alone. Second, the gap between the empirical literature and what audit practitioners actually deploy remains opaque: there is no widely-accepted measuring stick against which a new technique can claim concrete improvement.

By contrast, neighbouring fields have made disciplined use of public benchmarks for two decades. ImageNet and CIFAR [5, 19] drove a generation of progress in computer vision precisely because results could be reproduced and compared at near-zero friction. The Open Graph Benchmark [9] and follow-on benchmarks for graph anomaly detection [23, 31] have begun to provide the same service for graph machine learning. No equivalent reference exists in the accounting domain.

The obstacle is not principally technical: it is institutional. Real general ledgers cannot be released without exposing identifiable business activity, customer and vendor relationships, and personnel data. Even after anonymisation, joins against public records can re-identify entities; even after aggregation, the resulting data may no longer support the structural analyses that audit techniques depend on. The community has therefore been left with a choice between

empirical realism (private, irreproducible) and reproducibility (public, but typically toy-scale and unrealistic).

This paper takes the position that the choice is false—provided the synthetic side is held to a high empirical and statistical standard. We construct VynFi-Net, a synthetic reference benchmark, on top of the DataSynth generator [16] and an empirical study of 364 million journal-entry postings drawn from 155 client general ledgers [14]. The generator is open source [15], the configuration that produces the benchmark dataset is shipped in the same repository, and the edge-list export, anomaly labels, and baseline implementations are all reproducible end-to-end from a single command.

1.2 Importance of analysing journal entries

Journal entries are the elementary units of accounting truth. Every financial event in an enterprise—a sale, a goods receipt, a payroll run, a depreciation calculation, an intercompany allocation—is recorded as a journal entry consisting of two or more line items, the debits of which must equal the credits. Trial balances, subledger reports, financial statements, and consolidated group accounts are all derived structures: they are aggregations of, or projections from, the underlying journal-entry table. An auditor reasoning about an anomaly in any of these derived structures must, at the limit, trace the anomaly back to the journal entries that produced it.

This primacy of the journal entry is reflected in the audit standards. ISA 240 [12] explicitly requires the auditor to test journal entries for indicators of management fraud, and lists characteristics—unusual posting times, round-dollar amounts, manual postings to typically-automated accounts, period-end clustering—that have become standard targets for analytical procedures. ISO 21378 [13] standardises the schema in which journal entries are exported for audit data collection precisely because audit work products converge on this layer. Most recurring fraud typologies documented in the practitioner literature—split transactions, ghost employees, duplicate payments, period-cutoff manipulation, fictitious revenue—manifest first as patterns in journal entries.

The consequence for machine learning is that the journal-entry table is the right object of study. Aggregated views (trial balances, roll-up dashboards) discard the structure on which detection depends; narrower views (single-account histories) discard the cross-account relationships that distinguish, for example, a legitimate revenue posting from a fictitious one. A benchmark designed to drive methodological progress must therefore expose journal entries in their native granularity, with their full source-document and inter-account context.

1.3 Benefits of accounting-network analysis

Journal entries do not merely admit a network interpretation; they *are* a directed graph by construction. This is not a new observation: Ijiri [10] formalised the algebraic relationship between double-entry bookkeeping and the incidence matrix of a directed graph in 1965, and the same author later argued in his programmatic essay "The beauty of double-entry bookkeeping" [11] that the directed-graph structure—accounts as nodes, postings as directed edges from credit accounts to debit accounts—is the mathematical signature of double-entry as an information system, and the property that distinguishes it from single-entry record-keeping. Subsequent work

in the audit-analytics literature [1, 6, 21] has built on this foundation, treating the accounting graph as the primary object of study rather than the journal-entry table from which it is derived; the present author's earlier empirical study [14] reports the structural regularities of this graph at scale across 155 general ledgers.

The consequence is that an accounting network is not a derived view of the underlying postings: the postings *are* edges, the accounts *are* nodes, and the journal-entry table is a particular tabular serialisation of an object that is intrinsically graph-shaped. The accounting network therefore carries information that is not present in any single row of the row-level table.

First, network structure exposes recurring *motifs*. Routine operating cycles—procure-to-pay, order-to-cash, hire-to-retire—trace out characteristic edge patterns whose deviations are themselves diagnostic. A purchase order that triggers a goods receipt that triggers a vendor invoice that triggers a payment yields a predictable sequence of edges across well-defined account pairs; schemes such as duplicate payments, three-way-match overrides, and cutoff manipulation appear as distinctive perturbations of that sequence.

Second, network structure is the natural input for graph neural networks (GNNs). GNNs aggregate information across local neighbourhoods of the graph and have been shown to outperform non-relational baselines on a wide variety of structured-data tasks [7, 17, 33, 35]. On accounting graphs in particular, edge-level classification of suspect postings can exploit the company a posting keeps—the other edges incident to the same accounts, the same documents, or the same counterparties—in ways that flat tabular models cannot. GNN-based graph anomaly detection has been actively benchmarked on synthetic and public-domain graph data [23, 31], but *not* on accounting graphs of realistic size and structure.

Third, network analysis supports a class of statistical reasoning that is otherwise hard to formalise. Benford's law [25] characterises distributional properties of amounts, but the same postings additionally carry topological signatures—degree distributions, density of strongly-connected components, betweenness of central accounts—that fraud and error patterns alter. A reference benchmark that exposes both the row-level table and the induced network-edge list is therefore strictly more useful than one that exposes only the former.

1.4 Contributions

This paper makes the following contributions, organised in the order in which they appear in the remainder of the document.

1. **Empirical foundation.** We summarise—and revisit where appropriate—the empirical study of journal-entry line-item distributions on a corpus of 364 million postings drawn from 155 client general ledgers [14]. The recovered distribution informs the parameters that DataSynth [16] uses to generate accounting networks of realistic structure (Section 3).

2. **Network construction methods.** We present, with full mathematical statement, the five Cartesian-product methods (A through E) by which a balanced journal entry can be materialised as a set of debit $\rightarrow$ credit edges, and show their correspondence to the standard financial-statement-consolidation operation (Section 5).

3. **Reference graph schema.** We define a 13-column flat edge-list export (`graphs/je_network.csv`) with deterministic UUID v5 edge identifiers, joinable column-for-column against the journal-entry table and against the document-flow chain via a `predecessor_line_id` field (Section 6). The schema is sufficient to reproduce all baselines reported here without additional pre-processing.

4. **Benchmark design.** We open-source VynFi-Net: a fully-specified benchmark with three reproducible splits (temporal, entity, fraud-typology), a labelling discipline that surfaces over 130 anomaly subtypes at the edge level, and a leaderboard protocol that includes a leakage audit (Section 7).

5. **Strong baselines and initial evaluation.** We provide a baseline suite covering five GNN architectures, gradient-boosted trees, two classical anomaly detectors, and a rule-based reference, all trained from a single declarative configuration, and report initial numbers on the released splits (Section 8).

1.5 Paper organisation

Section 2 surveys the prior literature on accounting-anomaly detection, graph anomaly benchmarks, and synthetic-data generation for finance. Section 3 revisits the empirical foundations. Section 4 summarises the DataSynth generator that produces the benchmark dataset. Section 5 states the network construction methods. Section 6 describes the reference graph schema. Section 7 fixes the benchmark protocol. Section 8 reports baseline results. Section 9 interprets them and discusses threats to validity, and Section 10 lists open problems. Section 11 concludes.

2 Related Work

This section places VynFi-Net in three nearby literatures: machine learning for accounting and audit, graph anomaly-detection benchmarks in the wider machine-learning community, and synthetic-data generation for finance. We discuss each in turn and identify the gap that motivates the present work.

2.1 Machine learning for accounting and audit

Machine-learning approaches to accounting-anomaly detection date to the expert-system literature of the 1990s and have accelerated steadily over the last decade. West and Bhattacharya [34] survey the broader landscape of intelligent financial-fraud detection and emphasise the predominance of supervised methods on imbalanced data; Hilal et al. [8] update the survey for the deep-learning era and document a shift towards autoencoder, sequence-model, and graph-based approaches. Within accounting specifically, Schreyer et al. [28] demonstrated that deep autoencoder networks reconstruct routine general-ledger postings well and surface unusual entries through reconstruction-error scoring; Schreyer et al. [29] extend this with adversarial training to detect synthetic "accounting-deepfake" postings designed to evade autoencoder defences. Liang et al. [21] apply pattern-recognition methods to the bookkeeping graph itself, showing that subgraph anomalies can be detected when the journal-entry table is interpreted as

a network. Domain-specific work on VAT-compliance violations [20] and on continuous-auditing analytics [32] highlights the close relationship between detection technique and the particular accounting structure on which it operates. Guo et al. [6] formalise the topology of the accounting graph for audit and fraud-detection purposes and argue, in a position consonant with the present paper, that accounting-graph structure encodes information not present in the row-level table. Boersma [1] explores the use of complex-network methods in audit and offers a data-driven modelling perspective that complements the present work.

A persistent feature of this literature, however, is that empirical evaluation is conducted on private datasets—typically a single client's general ledger held under audit-confidentiality constraints—and the released code, when any, does not include the data on which the published numbers were computed. Cross-paper comparison is therefore not possible: a "state-of-the-art" method on one corporate ledger may underperform a simpler model on another, and the field has no standard way of detecting this from the published numbers alone.

2.2 Graph anomaly-detection benchmarks

Outside accounting, graph-based anomaly detection has converged on a small number of public benchmarks. The Open Graph Benchmark [9] provides standardised splits and evaluation protocols for node-, link-, and graph-level prediction tasks across citation, biological, and product graphs and has been widely adopted as a comparison frame for graph neural networks (GNNs). More recently, dedicated graph-anomaly benchmarks have emerged: Liu et al. [23] (GADBench) revisit supervised graph-anomaly detection across heterogeneous graph families, and Tang et al. [31] (BOND) survey unsupervised outlier-node detection on static attributed graphs. Both benchmarks have proved valuable in clarifying which methodological gains generalise across graph types and which are dataset-specific.

Neither GADBench nor BOND, however, includes accounting graphs of realistic structure. Accounting graphs differ from the citation, social, and product graphs that drive these benchmarks in three ways that matter for evaluation: (i) edges carry *signed monetary amounts* subject to a balance constraint ($\sum D = \sum C$ per posting), (ii) the underlying business processes (P2P, O2C, R2R) impose recurring *document-flow patterns* across edges, and (iii) the standards-driven categorical structure (account class, account sub-class, business process) is itself a strong predictive signal that no general graph carries by default. A benchmark that exposes these features explicitly is, to our knowledge, not available in the public-graph-benchmark space, and it is the gap VynFi-Net is designed to fill.

2.3 Synthetic data for finance

The third nearby literature is synthetic data generation for finance. Xu et al. [36] introduced CTGAN and TVAE for conditional tabular generation; Patki et al. [26] earlier presented the Synthetic Data Vault for hierarchical relational data; Solatorio and Dupriez [30] and Kotelnikov et al. [18] extended tabular generation to transformer- and diffusion-based architectures. In the finance domain, the BankSim simulator [24] and its successors have made transaction-level fraud-detection studies reproducible and have, in particular, given rise to the widely-used PaySim corpus.

These efforts share an important property with VynFi-Net: they generate data that is independently releasable, free of confidentiality constraints, and reproducible up to a known seed. They differ in a property that is decisive for accounting: VynFi-Net is designed not as a tabular sampler trained from real data, but as a *forward* generator that respects the hard accounting invariants ($\sum D = \sum C$ per posting, $A = L + E$ at consolidation, document-chain integrity for P2P/O2C flows) by construction [16]. Sample-based generators have no native mechanism to enforce these constraints; the forward-generation approach that DataSynth implements does, and it also exposes the underlying generative process to the user. This positioning is consonant with the broader argument in Cambria et al. [3] that knowledge-graph-style structured generation gives stronger reasoning purchase than purely sample-based methods.

3 Empirical Foundations

VynFi-Net is calibrated against an empirical study of journal-entry structure conducted on a large corpus of real general ledgers [14]. This section summarises the findings that drive the parameters of the DataSynth generator and the distributional checks reported in Section 8. All figures in this section are reproduced from Ivertowski [14] with permission of the author; the underlying client data are confidential and are not redistributed.

3.1 Corpus

The corpus comprises 155 general-ledger datasets (approximately 0.2% of the relevant population), drawn from a representative mix of industry sectors and covering each subject's full fiscal year of activity. Aggregate volumes are approximately 364 million journal entries and 2.4 billion line items. Datasets are classified into three size categories (Table 1): small ($<$1M lines per annum, 74 sets), medium (1–10M lines per annum, 47 sets), and large ($>$10M lines per annum, 34 sets).

Table 1: Classification of the source datasets by size. Source: Ivertowski [14].

	Small ($<$1M lpa)	**Medium** (1–10M lpa)	**Large** ($>$10M lpa)
Number of sets	74	47	34

3.2 Line-item distribution per journal entry

The empirical distribution of line items per journal entry is extremely concentrated. 92.60% of all postings carry between two and nine line items; the remaining mass tapers across orders of magnitude up to a maximum of 47 059 line items in a single posting. Table 2 reproduces the bin-level distribution. Within bin 1 (2–9 line items), Table 3 shows that two-line entries alone account for 60.68% of the corpus, with even-count entries $(2, 4, 6, \ldots)$ cumulatively dominating: roughly 88% of journal entries carry an even number of lines.

Table 2: Distribution of journal-entry line counts across the corpus. Source: Ivertowski [14], Table II.

Bin	Line items	Total JEs	Cum. coverage
1	2–9	338M	92.60%
2	10–99	23M	99.22%
3	100–999	2.80M	99.98%
4	1 000–9 999	63k	100.00%
5	$\geq$ 10 000	196	100.00%

Table 3: Detailed distribution within bin 1 (2–9 line items). Source: Ivertowski [14], Table III.

Line items	Total JEs	Coverage	Cum. coverage
2	221M	60.68%	60.68%
3	21M	5.77%	66.46%
4	61M	16.63%	83.09%
5	11M	3.06%	86.15%
6	12M	3.32%	89.47%
7	4M	1.13%	90.60%
8	7M	1.88%	92.48%
9	2M	0.42%	92.90%

3.3 Debit-credit symmetry

Within the corpus, 82% of journal entries have the same number of debit and credit line items; 11% have fewer credit than debit items and 7% vice versa. The dominance of even line counts follows directly from this symmetry, given that GAAP requires every posting to balance ($\sum D = \sum C$); the residual 12% with an odd line count corresponds to entries in which a single side (debit or credit) carries an aggregated counterpart of multiple line items on the other. These statistics motivate Method A as the default network-construction strategy in Section 5: any two-line balanced entry admits a unique 1-to-1 debit-credit edge, and any n-debit-and-n-credit balanced entry can be solved without decomposition.

3.4 Account population

The number of distinct general-ledger accounts populated in a single fiscal year ranges from 76 to 2 536 across the corpus, with mean 557 and median 367 (Ivertowski 14, Fig. 3). This range informs DataSynth's industry-pack expansion (manufacturing, retail, financial-services, healthcare, technology), which adds sub-account splits to base charts of accounts to bring synthetic chart sizes into the empirically observed range.

3.5 Implications for the benchmark

Three implications drive the design of VynFi-Net.

(i) Two-line entries dominate. Method A, the default 1-to-1 debit-credit edge construction, applies to roughly 60% of all postings and is the single most important construction step.

(ii) Multi-line entries are the long tail that matters analytically. Although bins 2–5 collectively cover only 7.4% of journal entries, they include the entries that are most frequently of audit interest—multi-line allocation entries, period-end accruals, and complex consolidation postings. Methods B–E from Section 5 exist precisely to handle these cases.

(iii) Even line counts are the norm; odd line counts are diagnostic. The empirical 88%/12% even/odd split, when combined with audit-relevant flags such as `is_post_close` and `document_type`, supplies a strong baseline against which structural anomalies can be measured.

The configuration that produces the released VynFi-Net dataset (Appendix A) reproduces these distributions within the tolerances reported in Section 8.

4 The DataSynth Generator

VynFi-Net is built on the DataSynth generator [15, 16], an open-source Rust framework for forward generation of coherent enterprise datasets. This section summarises the parts of DataSynth that the benchmark relies on; for the full architectural description we refer to the SSRN paper. No reader of this benchmark paper is expected to modify DataSynth; all that is needed to reproduce VynFi-Net is the configuration in Appendix A and the released DataSynth version pinned therein.

4.1 Architecture

DataSynth is a Cargo workspace of 17 active crates, of which five are directly relevant to VynFi-Net: `datasynth-core` (domain models and resource guards), `datasynth-config` (YAML configuration and validation), `datasynth-generators` (journal-entry, document-flow, sub-ledger, and anomaly generators), `datasynth-runtime` (orchestrator and output writers), and `datasynth-graph` (graph builders and the edge-list export that produces `graphs/je_network.csv`).

Determinism is provided by a single `ChaCha8` pseudo-random number generator seeded from the configuration; identifiers (entity codes, document numbers, line-level UUIDs, and edge-level UUIDs) are derived through stable hashing so that any two runs with the same seed and configuration produce identical output. A `DeterministicUuidFactory` discriminates UUIDs by generator type to avoid collisions between modules.

4.2 Coherence guarantees

The generator enforces a small number of hard accounting invariants *at construction time*, by design rather than by post-hoc validation:

- **Per-entry balance.** Every `JournalEntry` instance is constructed with $\sum D = \sum C$ to within the rounding tolerance of `rust_decimal`; the type system prevents construction of an unbalanced entry.
- **Document-chain integrity.** P2P chains (purchase order → goods receipt → vendor invoice → payment) and O2C chains (sales order → delivery → customer invoice → customer

receipt) maintain cross-document references; the corresponding journal entries inherit a stable `document_id` that the edge-list export consumes when wiring `predecessor_edge_id`.

- **Balance-sheet identity.** $A = L + E$ holds across every reporting period; subledger reconciliation reports are produced and checked.

These invariants—in addition to twenty-five further coherence checks shipped with the runtime—are what allow the released benchmark to support strong evaluations: a synthetic dataset on which trivial sanity checks fail does not justify methodological claims.

4.3 Statistical layer

Amounts are drawn from a configurable log-normal mixture (default three components: routine, significant, and major postings) with optional Benford-first-digit calibration; cross-field dependencies are induced by Gaussian, Clayton, Gumbel, Frank, or Student-t copulas; temporal dynamics include period-end clustering, business-day calendars for 15 regions, processing-lag modelling for event-to-posting delays, and intraday-volume profiles (morning spike, lunch dip, end-of-day rush). Industry profiles (retail, manufacturing, financial-services, healthcare, technology) preset these parameters so that a single switch in the configuration produces datasets with industry-appropriate distributional signatures.

4.4 Anomaly injection and labelling

DataSynth ships an injector covering more than 130 anomaly subtypes across five categories: fraud (e.g. fictitious revenue, split transactions, ghost employees, duplicate payments), error (e.g. duplicate entries, reversed amounts, wrong period or account), process (late posting, skipped approval, threshold manipulation), statistical (Benford violation, unusual amount, trend break), and relational (circular transaction, dormant-account activity). Every flagged journal entry passes through a behavioural bias module that applies weekend, round-dollar, off-hours, and post-close shifts with configurable probabilities, so that surface features used by ISA 240-style heuristics are exercised [12].

The labels propagate through the edge-list export: an edge is flagged with `is_fraud=true` if either of its endpoints is on a line of an entry flagged `is_fraud=true`, and similarly for `is_anomaly`. This propagation rule is the basis of the edge-level binary and multi-class anomaly-detection tasks (Section 7).

4.5 ISO 21378 alignment

DataSynth populates the ISO 21378 [13] account-class and account-sub-class fields on every `GLAccount`, with a total mapping from the internal `AccountSubType` enumeration into the standard's three-level (`Type`/`Class`/ `Sub-Class`) classification. This makes the released benchmark immediately compatible with audit data-collection tooling that already speaks ISO 21378 and supports cross-jurisdictional splits in future versions of VynFi-Net.

5 Network Construction

This section makes precise the mapping from the row-level journal-entry table to a directed account-level graph. The treatment follows the formulation in Ivertowski [14] but is presented self-contained.

5.1 Setup and notation

Let $\mathcal{A}$ be the set of general-ledger accounts in the chart of accounts. A *journal entry* J is a balanced pair (D, C) of multisets of line items

$$D = \{(a_1^D, d_1), \ldots, (a_U^D, d_U)\}, \quad C = \{(a_1^C, c_1), \ldots, (a_V^C, c_V)\},$$

with $a_u^D, a_v^C \in \mathcal{A}$ and $d_u, c_v \in \mathbb{R}_{>0}$ satisfying the balance constraint

$$\sum_{u=1}^{U} d_u \;=\; \sum_{v=1}^{V} c_v. \tag{1}$$

We refer to $U = |D|$ and $V = |C|$ as the debit and credit *cardinalities*. Bin 1 of Section 3.2 corresponds to $U + V \in \{2, \ldots, 9\}$ and accounts for 92.60% of empirical postings.

The *accounting graph* associated with a collection of journal entries over a period $[t_0, t_1]$ is the directed multigraph

$$G(t_0, t_1) \;=\; \big(A(t_0, t_1),\, E(t_0, t_1)\big), \tag{2}$$

where the node set $A(t_0, t_1) \subseteq \mathcal{A}$ contains every account that carries a non-zero balance over the period and the edge set $E(t_0, t_1)$ contains a weighted, signed, time-stamped edge for every (a^D, a^C) pair generated by some journal entry posted in $[t_0, t_1]$. When edges between the same ordered account pair are aggregated, the period-level edge weight is

$$E_{(a,b)}(t_0, t_1) \;=\; \sum_{n=1}^{|\Omega_{(a,b),[t_0,t_1]}|} \omega_n, \tag{3}$$

where $\Omega_{(a,b),[t_0,t_1]}$ is the multiset of individual transaction flows from a to b posted in the period and ω_n is the monetary amount of the n-th such flow. Equation (3) reproduces Ivertowski [14] Eq. (2).

The construction of the per-entry edge set is the central object of this section. Five methods (A through E) are defined; the choice of method depends on the cardinalities (U, V) and on whether the line-item amounts are pairwise distinct. Methods are tried in order; the first that succeeds is the one that materialises the edge set for the entry. Algorithm 1 states the dispatch.

5.2 Method A: 1-to-1 mapping

Method A applies when exactly one debit and one credit line ($U = V = 1$, hence $U + V = 2$). By the balance constraint (1) the single debit amount equals the single credit amount, and a

single edge (a_1^D, a_1^C) with weight $d_1 = c_1$ is materialised:

$$E_J^A \;=\; \{\,(a_1^D, a_1^C, d_1)\,\}.$$

Method A is exact: every line item maps bijectively to the unique debit-credit pair, and the *confidence* of the inferred edge is 1 (Table 4). Per Section 3.2, Method A applies to 60.68% of postings in the corpus.

5.3 Method B: n-to-n mapping

Method B applies when $U = V \geq 2$, that is, when an equal number of debit and credit line items appear. Method B takes the Cartesian product of debit and credit lines:

$$E_J^B \;=\; \{\,(a_u^D, a_v^C, w_{uv}) : u \in \{1, \ldots, U\}, v \in \{1, \ldots, V\}\,\},$$

where the weight w_{uv} is $\min(d_u, c_v)$ when the line-item amounts are pairwise distinct (so the matching is forced), and $\min(d_u, c_v)$ scaled by a confidence factor when they are not. The confidence assigned to a non-distinct edge is $1/n_i$ where n_i is the number of indistinguishable values the i-th line item takes (Table 4). Method B preserves total monetary flow ($\sum_{u,v} w_{uv}$ equals the balanced amount) but does not in general preserve the per-line allocation when amounts collide.

5.4 Method C: n-to-m mapping

Method C applies when $U \neq V$ and the entry's line-item amounts admit a partition that satisfies the balance constraint on both sides. When $V > U$ (more credit lines than debit lines), this is the classical problem of partitioning the V credit amounts into U groups whose sums match the U debit amounts. The number of admissible partitions is

$$a(n, m) \;=\; \frac{n!}{m!} \sum_{j=0}^{m} (-1)^{m-j} \binom{m}{j} j^n, \tag{4}$$

reproducing Eq. (3) of Ivertowski [14] with $n = V$ and $m = U$. When this combinatorial count exceeds a configurable budget—which it does rapidly for large n, m—Method C is abandoned in favour of Methods D or E.

When the partition exists and is admissible, Method C materialises one edge per (debit-line, credit-group) pair with weight equal to the group sum. Confidence is 1 when the partition is unique and $1/n_i$ otherwise (Table 4).

5.5 Method D: composition to higher aggregate

Method D handles the case where Methods B and C fail by *composing* line items up to a higher aggregate of the chart of accounts (account sub-class `ASC` or account class `AC`). Aggregation is exact—debits and credits sum unchanged—and may collapse the cardinality mismatch ($U \neq V$) into the equal case ($U = V$), in which event Method B can then be applied at the aggregated level. Method D re-runs the dispatch chain (Methods A, B, C, in that order) on the aggregated

entry; if it succeeds, the resulting edges connect *aggregate* nodes rather than individual accounts. Confidence is 1 when the aggregated entry admits an exact lower-level solution (Table 4).

5.6 Method E: decomposition with shadow line items

Method E is the residual fallback when no exact solution exists at any aggregation level. It decomposes credit-side amounts into shadow line items that match debit-side amounts, materialising edges with confidence $1/n^b$ where n^b is the number of decomposition steps. Method E always produces a result and so guarantees that the algorithm terminates with a valid edge set; it is, however, the lowest-confidence method, and its prevalence on a particular dataset is itself a quality signal.

5.7 Confidence

Table 4: Per-method edge confidence. Source: Ivertowski [14], Table V. n_i denotes the count of the i-th line-item value; n^b denotes the count of decomposition steps.

Method	**Case**	**Confidence**
A	($U = V = 1$, distinct)	1.00
B	($U = V \geq 2$, distinct)	1.00
B	($U = V \geq 2$, not distinct)	$1.00/n_i$
C	($V > U$, distinct)	1.00
C	($V > U$, not distinct)	$1.00/n_i$
C	($V < U$)	$1.00/n^b$
D	(any case, exact aggregation)	1.00
E	(decomposition fallback)	$1.00/n^b$

The journal-entry-level confidence is the product of the line-level confidences over all edges produced for the entry, consistent with the treatment in Ivertowski [14]. This per-entry confidence is exposed on every row of `graphs/je_network.csv` in the `confidence` column (Section 6).

5.8 Dispatch

Algorithm 1: Network generation (after Ivertowski [14], Algorithm 1).

Input: Journal entries $J_1, \ldots, J_N$; chart of accounts aggregator $\mathcal{M}$.

Output: Edge sets $\mathrm{NR}_1, \ldots, \mathrm{NR}_N$.

for $n \leftarrow 1$ **to** N **do**
  $U \leftarrow |D(J_n)|$; $V \leftarrow |C(J_n)|$
  if $U + V = 2$ **then**
    $\mathrm{NR}_n \leftarrow \text{Method A}(J_n)$
  else if $U = V$ **then**
    $\mathrm{NR}_n \leftarrow \text{Method B}(J_n)$
  else if Method C(J_n) is feasible **then**
    $\mathrm{NR}_n \leftarrow \text{Method C}(J_n)$
  else
    $J_n^{\mathrm{ASC}} \leftarrow \text{Method D}(J_n, \mathcal{M}, \texttt{ASC})$
    if *dispatch on* J_n^{ASC} *succeeds* **then**
      $\mathrm{NR}_n \leftarrow$ dispatch result
    else
      $J_n^{\mathrm{AC}} \leftarrow \text{Method D}(J_n, \mathcal{M}, \texttt{AC})$
      if *dispatch on* J_n^{AC} *succeeds* **then**
        $\mathrm{NR}_n \leftarrow$ dispatch result
      else
        $\mathrm{NR}_n \leftarrow \text{Method E}(J_n)$
return $\mathrm{NR}_1, \ldots, \mathrm{NR}_N$

5.9 Default in the released benchmark

For VynFi-Net the default construction is Method A on 1-to-1 entries and Method B on n-to-n entries with $U = V$. Methods C–E remain available for ablation studies where a reader wants to test sensitivity to construction strategy; they are activated by the `network_construction` option in the released configuration (Appendix A). This default reproduces the edge counts reported in Section 8; alternative configurations produce different counts by construction and hence require a re-baselining of all reported numbers.

5.10 Equivalence at the period level

Although Methods A through E differ in how they distribute monetary flow across edges *within* a single posting, they are *equivalent at the period level* in the sense of Equation (3): for any two methods M and M' in the family, the aggregated edge weight $E_{(a,b)}(t_0, t_1)$ is the same when summed over all postings in the period. This follows from the balance constraint (1): the total monetary flow into and out of any account is determined by the line items of the entries, not by the way those line items are paired. The methods therefore correspond to different *micro-level* resolutions of the same *macro-level* consolidation, in direct analogy to the financial-statement consolidation operation in which subsidiary ledgers are aggregated to group ledgers.

6 Reference Graph and Schema

This section fixes the on-disk schema of the released benchmark. The single artefact that a downstream user needs in order to train and evaluate any of the baselines reported in Section 8 is `graphs/je_network.csv`; companion files are available for joining additional features back from the row-level journal-entry table.

6.1 File layout

The released benchmark archive contains the following top-level files:

- `graphs/je_network.csv` — the flat edge list, one row per debit-credit edge produced by the Section 5 dispatch. This is the primary input to all baselines.
- `journal_entries.csv` — the row-level 42-column journal-entry table, one row per line item, used to recover entry-level features when needed.
- `coa.csv` — the chart of accounts with ISO 21378 `Type`, `Class`, and `Sub-Class` fields populated alongside the internal `AccountType` classification.
- `period_close/trial_balances/` — per-period trial balances for spot-checking distributional tests.
- `labels/` — per-entry anomaly labels (`anomaly_labels.csv`, `fraud_labels.csv`, `quality_labels.csv`); already projected onto the edge list, but exposed separately for users who wish to re-label.

The full set of ∼60 output directories produced by DataSynth is available in the same archive for users who require non-anomaly features (master data, document flow, sub-ledger, intercompany eliminations, etc.); these are documented in [16] and are not strictly required for the baselines reported here.

6.2 Edge-list schema

Table 5 fixes the 13 columns of `graphs/je_network.csv`. All identifiers are UTF-8 strings; all amounts are serialised as decimal strings (no IEEE 754) to preserve exact precision; dates use ISO 8601 (`YYYY-MM-DD`).

Table 5: Schema of `graphs/je_network.csv`.

Column	Type	Description
`edge_id`	UUID v5	Deterministic identifier; UUID v5 of (document_id, debit.line_number, credit.line_number).
`document_id`	string	Source document identifier (e.g. `PO-2024-000123`). `Manual` when no source document.
`posting_date`	ISO date	Posting date of the underlying journal entry.
`from_account`	string	Debit (source) general-ledger account number.
`to_account`	string	Credit (destination) general-ledger account number.
`from_line_id`	UUID v5	Stable line-level identifier of the debit line.
`to_line_id`	UUID v5	Stable line-level identifier of the credit line.
`amount`	decimal	Edge weight in the entry's currency, in absolute value.
`confidence`	decimal	Per-entry confidence per Table 4; 1.00 for Methods A, B-distinct, C-distinct, and D.
`predecessor_edge_id`	UUID v5	Edge in the immediately preceding journal entry of the same document chain matching on `from_account`; empty for chain heads and for journal entries outside any chain.
`business_process`	enum	One of `P2P`, `O2C`, `R2R`, `H2R`, `A2R`, `S2C`, `MFG`, `BANK`, `AUDIT`, `Treasury`, `Tax`, `Intercompany`, `PROJECT`, `ESG`.
`is_fraud`	bool	Edge inherits the `is_fraud` flag of the underlying journal entry.
`is_anomaly`	bool	Edge inherits the broader `is_anomaly` flag, covering fraud, error, process, statistical, and relational categories (Section 4.4).

6.3 Deterministic edge identity

The `edge_id` field is the UUID v5 of the triple (document_id, debit.line_number, credit.line_number) under a fixed namespace seeded by DataSynth's `DeterministicUuidFactory`. Two consequences follow. First, two runs of the generator with the same seed and configuration produce identical `edge_id` values for every edge, and the released benchmark is therefore reproducible at the edge identifier level. Second, an edge identifier remains stable under irrelevant rearrangements (re-ordering of unrelated postings, re-numbering of unrelated documents) but *changes* when any of the three components changes; this is the property required for cross-version differential analysis.

6.4 Joinability

The schema is joinable column-for-column against the row-level journal-entry table via (document_id, from_line_ and (document_id, to_line_id). A typical query pattern, written in SQL shorthand, is:

```
SELECT  e.*, j_d.account_class, j_d.account_sub_class,
             j_c.is_post_close
FROM    je_network e
```

```
LEFT JOIN journal_entries j_d
       ON j_d.line_id = e.from_line_id
LEFT JOIN journal_entries j_c
       ON j_c.line_id = e.to_line_id
WHERE   e.posting_date BETWEEN '2024-10-01' AND '2024-12-31';
```

This recovers the full 42-column entry context for any edge without requiring duplication of features in the edge list.

6.5 Predecessor chain

The `predecessor_edge_id` field traces booking chains across documents. For a payment posting that clears an open vendor invoice, the predecessor edge is the corresponding AP_CONTROL → VENDOR_AUX edge of the invoice posting; for a customer receipt clearing an open AR balance, the predecessor edge is the corresponding AR_CONTROL → CUSTOMER_AUX edge. Heads of chains (purchase orders, sales orders, manual postings) carry an empty `predecessor_edge_id`. This field allows the construction of *document-chain features* (length of incoming chain, expected vs. observed time between adjacent postings, etc.) without requiring a separate document-flow export to be parsed.

6.6 Alternate exports

The same edge list is produced in three additional formats for users who prefer typed graph artefacts:

- PyTorch Geometric (`.pt`): node and edge tensors plus edge-level labels, ready to load into a `torch_geometric.data.Data` object.
- Neo4j: CSV node and relationship files plus a Cypher `LOAD CSV` script.
- DGL: a serialised `DGLGraph` (binary) with edge labels.

These exports are produced from the same edge-list source and share `edge_id` values with the CSV file; switching format does not alter any baseline numbers reported in Section 8.

7 Benchmark Design

This section fixes the VynFi-Net protocol: the tasks that submissions target, the splits over which they are evaluated, the metrics reported, and the reproducibility envelope under which a submission's numbers are accepted. All elements of the protocol are testable from the released artefacts alone, without requiring any private data or any non-public tooling.

7.1 Tasks

VynFi-Net defines three edge-level prediction tasks over `graphs/je_network.csv`:

- **T1: Binary anomaly classification.** Predict `is_anomaly` for each edge in the test split. The positive class includes all five categories (fraud, error, process, statistical, relational). T1 is the headline task and the one against which leaderboard rank is computed.
- **T2: Multi-class anomaly classification.** Predict one of {`none`, `fraud`, `error`, `process`, `statistical`, `relational`} for each edge. T2 supports per-category analysis and is the natural target for class-conditional fraud-detection methods.
- **T3: Rare-typology detection.** A subset of T2 in which only the rarest typology classes (cumulatively the bottom 5% of labelled edges) appear in the test split. T3 is the hardest task: even strong baselines on T1 and T2 fall back close to base-rate performance under T3.

For each task the prediction is per-edge (i.e. per row of the edge-list CSV); models that operate on the entry table, on the document chain, or on alternate graph projections are welcome, provided the final output is an edge-level score.

7.2 Splits

Three reproducible splits are released with VynFi-Net. They are not mutually exclusive: a method's performance on each is reported separately, and the leaderboard records all three.

- **Temporal.** The last three fiscal months of the generated period are held out as test; the preceding nine months form the training pool, with the last 20% of the training pool reserved as a validation slice. This split measures generalisation across time and is the most common real-world deployment scenario.
- **Entity.** Ten percent of subsidiaries (sampled uniformly without replacement) are held out as test; all postings of held-out subsidiaries are excluded from training. This split measures generalisation to entities unseen during training.
- **Fraud-typology.** Two of the five anomaly categories (sampled at random and fixed in the released splits file) are held out as test *positives*; the training set sees only the other three categories together with negatives. This split measures generalisation across fraud typologies and is the basis for T3.

The splits are materialised as CSV files (`splits/temporal_train.csv` etc.) listing the `edge_id` of every edge in each fold. No edge appears in both the train and test fold of the same split.

7.3 Metrics

All tasks are evaluated under heavy class imbalance (positive rate on the order of 1–5% depending on configuration), which makes precision-recall metrics more informative than ROC-based ones. Primary and secondary metrics are reported for every submission:

- **Primary.** AUC-PR (area under the precision–recall curve). Reported with a 95% bootstrap confidence interval.

- **Secondary.** AUC-ROC, F_1 at the threshold that maximises validation F_1, precision at $k = 50, 100, 500$, and recall at the top $1\%, 5\%, 10\%$ of scored edges.
- **Per-typology breakdown.** For T2 and T3, AUC-PR is also reported per anomaly category.

The leaderboard ranks methods by primary AUC-PR on the temporal split for T1; the entity- and typology-split numbers are reported alongside but do not enter the headline rank.

7.4 Leakage audit

A standing risk on any benchmark constructed from a single generated dataset is that information specific to a held-out posting leaks into the training pool through a different feature or row. VynFi-Net addresses this risk with an explicit leakage audit, the script for which is part of the release. The audit checks the following:

- *Document overlap.* No edge in the test split shares a `document_id` with any edge in the train split.
- *Predecessor closure.* No edge in the test split has a `predecessor_edge_id` pointing to an edge in a different split. This rule is what enforces document-chain integrity at the split boundary.
- *Entity-time coverage.* Under the entity split, no held-out subsidiary appears in any aggregated feature (consolidation entries, intercompany pairs) that would leak test-time data into training.

A submission that opts to engineer features from companion files (e.g. document-flow tables, sub-ledger reports) is required to re-run the audit on its feature-construction script.

7.5 Reproducibility envelope

Every submission must record the following metadata in the leaderboard entry:

- the DataSynth version (git tag or release number) used to produce the data;
- the configuration hash (SHA-256 of the canonical YAML);
- the ChaCha8 seed under which the dataset was generated;
- the model code repository and commit hash;
- the random seed(s) used in training.

Two submissions whose first three values agree are required to report identical edge-list contents; this is enforced by a checksum the audit script computes. Two submissions whose model code is identical and whose training seeds match are required to report identical predictions.

7.6 Submission protocol

A submission consists of the following artefacts, packaged in a single archive:

- per-task prediction files (`predictions/T1.csv` etc.), each with two columns `edge_id` and `score` (and an additional `predicted_class` column for T2/T3);
- the metadata block listed in Section 7.5;
- a description of the training pipeline of length not exceeding two pages, in PDF, sufficient for an independent reader to reproduce the result.

The release ships a scoring harness (`tools/bench_score.py`) that consumes a submission archive and emits the full metric table. Leaderboard entries are accepted on the basis of harness output; private re-evaluations are not required, but are encouraged for reviewers who wish to spot-check.

8 Empirical Evaluation

This section reports the headline results of the released configuration of VynFi-Net. Numerical entries marked "*[pending training run]*" will be filled in from the v1.0 release training runs; the methodology, baselines, and evaluation harness are fully specified here.

8.1 Released configuration

The released v1.0 configuration of VynFi-Net produces a twelve-month synthetic enterprise dataset under the manufacturing industry profile (Section 4). Headline volumes are summarised in Table 6; the verbatim YAML is reproduced in Appendix A.

Table 6: Released configuration v1.0: headline volumes. Edges are produced by Method A on 1-to-1 entries and Method B on n-to-n entries; counts are deterministic given the configuration seed.

Period	FY2024 (12 months)
Industry profile	Manufacturing
Subsidiaries	25
Chart-of-accounts size (mean)	∼520 accounts
Journal entries	*[pending training run]*
Edge count (`je_network.csv`)	*[pending training run]*
Anomaly rate (T1 positive)	∼3.5%
Fraud rate (subset)	∼1.0%

The fraud-rate split between line-level and document-level fraud follows the calibration formula of [16]: `fraud_rate` = 0.020, `document_fraud_rate` = 0.050, `propagate_to_lines` = true, yielding the ∼3.5% headline.

8.2 Baselines

We provide a baseline suite that spans the methodological space of edge-level anomaly detection. All baselines train from a single declarative configuration (`baselines/config.yaml`); the hyperparameter tables for each are listed in Appendix C.

- **Graph neural networks.** GCN [17], GraphSAGE [7], GAT [33], GIN [35], RGCN [27]. All are trained as edge-classification models with two 128-dimensional hidden layers; node features are constructed from account-class one-hots, account-degree statistics, and a normalised account-frequency embedding.
- **Tabular gradient boosting.** XGBoost [4] on a feature vector composed of the 13 edge columns plus aggregated neighbour statistics (mean / max / std of incident-edge amounts and same-document edge counts).
- **Classical anomaly detectors.** Local Outlier Factor [2] and Isolation Forest [22] on the same feature vector as XGBoost, used unsupervised and scored against the ground-truth labels.
- **Rule-based reference.** An ISA 240-style rule set [12]: round-dollar amounts, weekend or off-hours postings, post-close postings, manual postings to typically-automated accounts. This baseline is the fairest non-ML reference for "what an audit programme catches today".

8.3 Hyperparameter protocol

For each baseline we use the validation slice of the temporal split to select hyperparameters. We do not perform a full grid search; we report the single configuration shipped with the release plus, for the GNN family, a learning-rate sweep over $\{10^{-3}, 5\times10^{-4}, 10^{-4}\}$ and a depth sweep over $\{2, 3\}$ layers. The fixed configuration is what the release trains; the sweeps appear only in the ablation Section 8.6.

8.4 Headline results

Table 7 reports primary AUC-PR on T1 for each split. Entries marked *[pending]* will be supplied in the v1.0 release.

Table 7: T1 binary anomaly detection: primary AUC-PR by split. Higher is better. 95% bootstrap CIs in parentheses. *[All values pending v1.0 training run.]*

Method	Temporal	Entity	Typology
Rule-based reference	*[pending]*	*[pending]*	*[pending]*
Local Outlier Factor	*[pending]*	*[pending]*	*[pending]*
Isolation Forest	*[pending]*	*[pending]*	*[pending]*
XGBoost	*[pending]*	*[pending]*	*[pending]*
GCN [17]	*[pending]*	*[pending]*	*[pending]*
GraphSAGE [7]	*[pending]*	*[pending]*	*[pending]*
GAT [33]	*[pending]*	*[pending]*	*[pending]*
GIN [35]	*[pending]*	*[pending]*	*[pending]*
RGCN [27]	*[pending]*	*[pending]*	*[pending]*

We expect, on the basis of preliminary internal runs, that the ranking on the temporal split groups roughly into three tiers: relational-aware GNNs (RGCN, GAT) at the top, attention-free GNNs and XGBoost in a competitive middle band, and the unsupervised classical detectors and the rule-based reference forming the floor. We expect the entity-split numbers to lag the temporal numbers by a small margin and the typology-split numbers to lag substantially—this gap is itself one of the headline findings the benchmark is designed to surface.

8.5 Per-typology breakdown

Table 8 reports T2 per-typology AUC-PR on the temporal split. We anticipate, again from preliminary runs, that fraud is the most learnable category (it is also the category with the most distinctive behavioural-bias signature), followed by error and process anomalies; statistical and relational categories are harder and benefit most from network features.

Table 8: T2 multi-class AUC-PR on the temporal split, by anomaly category. *[All values pending v1.0 training run.]*

Method	Fraud	Error	Process	Statistical	Relational
Rule-based reference	*[pending]*	*[pending]*	*[pending]*	*[pending]*	*[pending]*
XGBoost	*[pending]*	*[pending]*	*[pending]*	*[pending]*	*[pending]*
Best GNN	*[pending]*	*[pending]*	*[pending]*	*[pending]*	*[pending]*

8.6 Ablations

The release supports three planned ablation experiments, each of which a downstream user can re-run from configuration:

- **Network depth.** Hold the GNN family fixed and vary the number of message-passing layers in $\{1, 2, 3, 4\}$; report AUC-PR on the temporal split.
- **`predecessor_edge_id` feature.** Re-train the best GNN with and without document-chain features derived from `predecessor_edge_id`; report the delta AUC-PR as evidence for

the value of chain context.

- **Structure-only vs. attribute-only.** Train two additional XGBoost variants: one on edge-list structure (no amounts, no dates, no flags) and one on edge attributes only (no graph features); both are reported alongside the full XGBoost as a decomposition of the signal.

8.7 Cross-typology generalisation

We also expect the typology-hold-out split to be the hardest of the three. Initial runs on a near-final configuration showed all baselines falling within a narrow AUC-PR band on T3—substantially below their T1 performance on the same data. This finding, which we will quantify in the v1.0 release, is a candidate driver for methodological work on *class-conditional* fraud detection that does not currently dominate the literature.

9 Discussion

We use this section to draw out the methodological consequences of the results in Section 8, to call out the most pressing threats to validity, and to position VynFi-Net in the broader research economy of audit analytics.

9.1 What baselines learn, and what they do not

The expected gap between the temporal and typology splits (Section 8.7) is the single most methodologically suggestive feature of the benchmark. Temporal generalisation is comparatively easy: held-out months draw from the same anomaly categories that the model has already seen during training, and the relevant signal is largely *within distribution*. Typology generalisation is harder by design: the held-out anomaly categories are absent from the training pool, and the model must infer that a posting is anomalous on the basis of *what is normal*, not on the basis of what previously labelled positives looked like.

Strong GNN baselines are, in our preliminary experience, much more sensitive to this distinction than tabular baselines. GNNs that do well on the temporal split frequently do not preserve their margin on the typology split, suggesting that they are exploiting class-specific structural cues rather than building a robust representation of normality. This is a candidate driver for methodological work on *anomaly definitions* that do not rely on fully supervised typology coverage—auto-encoders trained on clean training pools, contrastive losses that pair postings against their document-chain predecessors, or open-set detectors that explicitly admit unseen categories at test time.

9.2 Threats to validity

Synthetic-to-real transfer. The most serious threat to the validity of any synthetic benchmark is that methods which dominate on synthetic data fail to transfer to real client ledgers. VynFi-Net is calibrated against a 364M-posting empirical study (Section 3), and the released configuration reproduces the line-item, account-population, and temporal distributions reported

there. This calibration is necessary but not sufficient: distributional fidelity at the marginal level does not guarantee fidelity at the joint level, and we make no claim that a method's VynFi-Net score directly predicts its score on a client ledger. The role of VynFi-Net is to provide a *reference frame* on which methods can be compared, ablated, and reproduced; the validation against client data remains a separate exercise that practitioners must conduct themselves and under their own confidentiality protocols.

Anomaly definitions. The labels in `labels/anomaly_labels.csv` are determined by the generator's anomaly-injection module (Section 4.4); they reflect the categories the module is configured to inject and the propagation rule from entries to edges. A method that targets a different operational definition of "anomaly"—for example, a definition centred on control-failure rather than on fraud—should expect to see lower numbers on VynFi-Net than its target definition would suggest, and should report this gap explicitly when claiming applicability.

Single industry profile. The released v1.0 configuration is manufacturing-only. Cross-industry generalisation is not something the v1.0 benchmark numbers can measure. The configuration interface supports the four other industry profiles shipped with DataSynth, and we expect to release multi-industry splits in v1.1.

9.3 Confidentiality, reproducibility, and the role of public benchmarks in audit research

The principal motivation for VynFi-Net is the observation (Section 1.1) that a research community cannot accumulate methodological progress on private, irreproducible datasets. This observation cuts in two directions that are worth separating.

First, public benchmarks do not displace empirical work on real client data. An audit method that wins a leaderboard on VynFi-Net is not, by that fact alone, ready for production deployment; it must still be validated against the client ledger it will run against, under the auditor's own quality controls. A reference benchmark is a coordination device that lets methods be compared and ablated; it is not a substitute for client engagement.

Second, public benchmarks lower the cost of method development for audit practitioners. An audit team that wishes to evaluate a new fraud-detection technique against the published state of the art no longer needs to license and replicate a corpus or to negotiate data-sharing agreements; the team can run the technique against VynFi-Net and read the leaderboard. This is a particularly large gain for smaller audit firms, in-house audit functions, and academic researchers, whose access to client data is structurally more restricted than that of the largest firms.

9.4 Implications for practitioners

For practitioners specifically: VynFi-Net is a measuring stick. It does not—and is not designed to—tell a CAE or audit methodology lead which detection technique to deploy on next year's engagement. It does provide a defensible answer to the question "how does technique X compare to the state of the art on equivalent data?" When a vendor claims that a model achieves AUC-PR $= 0.X$, the practitioner can now ask whether the same model achieves the same

AUC-PR on VynFi-Net, and whether the model's performance on the typology-hold-out split is comparable. A negative answer is informative.

10 Future Work

We close with a list of extensions that the present release does not cover. Each is supported by a configuration switch in DataSynth, an export already produced by the runtime, or a related artefact in the repository, and so represents a relatively low-friction path from v1.0 of VynFi-Net to a richer benchmark.

10.1 Multi-period and rolling-window benchmarks

The v1.0 temporal split holds out the last three fiscal months of a single year. A more demanding regime is the rolling-window protocol that mirrors continuous-auditing practice: the model is re-trained at the close of each month on all data up to that point and evaluated on the following month, repeated across the year. This regime exposes whether a method's accuracy degrades under *distribution shift within* the period—a more realistic deployment scenario than a single hold-out. The required configuration changes are local to the splits files; no new generator features are needed.

10.2 Cross-jurisdictional comparability

DataSynth's ISO 21378 alignment (Section 4.5) populates account-class and account-sub-class fields that are common across jurisdictions but does so on a single chart of accounts at a time. A natural next step is to construct splits that hold out entire jurisdictions—training on US-GAAP and IFRS, evaluating on French- and German-GAAP charts—and to report the resulting drop in performance as a measure of cross-jurisdictional transfer. This is exactly the kind of question the standardised account-class fields were designed to support.

10.3 Counterfactual evaluation

The DataSynth counterfactual engine produces, for any base generation, a paired *counterfactual* dataset in which a specified perturbation (an anomaly removed, a different control in place, a different subsidiary structure) propagates through the causal DAG. Pairing VynFi-Net edges with their counterfactuals yields a strictly stronger evaluation: a method should not only flag the anomalous edge in the base run, but also *not* flag the corresponding edge in the counterfactual run where the anomaly is absent. We anticipate releasing counterfactual pairs in v1.1.

10.4 Real-data calibration loop

The empirical foundations of the generator (Section 3) were established on a 2024 corpus. The corpus is not static—new client engagements continually produce additional ledgers—and so the calibration of the generator should be revisited periodically. We propose to publish, with each minor version of VynFi-Net, a refresh of the empirical Tables 1–3 and a delta against the

previous calibration; benchmark numbers are versioned to specific calibrations so that changes are traceable.

10.5 Beyond anomaly detection

The benchmark schema is a general-purpose accounting graph and supports tasks adjacent to anomaly detection: control testing (does the method correctly identify the controls implicated by a given posting?), account-relationship discovery (does the method recover the canonical chart-of-accounts hierarchy from the edge list?), and cutoff testing (does the method correctly classify postings near period boundaries as in-period or out-of-period?). These tasks would be released as separate "challenges" on the same underlying graph—a model trained for anomaly detection could be evaluated against them zero-shot, and a model trained jointly across challenges might benefit from the additional supervision.

11 Conclusion

Empirical machine-learning research on accounting-anomaly detection has matured to the point where standardised, reproducible benchmarks are overdue. The community has accumulated a respectable toolkit—deep autoencoders, adversarial detectors, gradient boosting, graph neural networks—but no shared evaluation frame on which to compare them, because the underlying data is held under audit-confidentiality constraints that public release does not respect. Synthetic data, when it is held to a high empirical and statistical standard, dissolves that constraint.

VynFi-Net is an attempt to construct exactly such a shared frame. It is calibrated against an empirical study of 364 million journal-entry postings drawn from 155 general ledgers [14]; it is generated by the open-source DataSynth framework [15, 16] under a single declarative configuration; it exposes a 13-column flat edge list whose schema is fixed and joinable against the row-level journal-entry table; and it ships with three reproducible splits, an explicit leakage audit, and a baseline suite spanning graph neural networks, gradient boosting, classical detectors, and an ISA 240-style rule-based reference.

VynFi-Net does not displace empirical work on real client data. Its role is complementary: a public reference against which methods can be developed, ablated, compared, and reproduced without the confidentiality concerns that block public release of the underlying ledgers. We anticipate that this role will become more useful as the benchmark matures: rolling-window splits, cross-jurisdictional configurations, counterfactual pairs, and adjacent challenges (control testing, cutoff testing) are all within reach of the present infrastructure.

We invite the community to use VynFi-Net, to submit to the leaderboard, and to file issues against the configuration where the empirical calibration drifts away from contemporary client practice. The benchmark is most valuable when it is held collectively to the standard the field needs.

References

[1] Marcus Boersma. *Complex networks in audit: A data-driven modelling approach.* PhD thesis, Universiteit van Amsterdam, 2024.

[2] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. LOF: Identifying density-based local outliers. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 93–104, 2000.

[3] Erik Cambria, Shaoxiong Ji, Shirui Pan, and Philip S. Yu. Knowledge graph representation and reasoning. *Neurocomputing*, 461:494–496, 2021.

[4] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[5] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009.

[6] Kelvin H. Guo, Xiang Yu, and Carla L. Wilkin. A picture is worth a thousand journal entries: Accounting graph topology for auditing and fraud detection. *Journal of Information Systems*, 36(2):53–81, 2022.

[7] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[8] Wassim Hilal, S. Andrew Gadsden, and John Yawney. Financial fraud: A review of anomaly-detection techniques and recent advances. *Expert Systems with Applications*, 193:116429, 2022.

[9] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[10] Yuji Ijiri. On the generalized inverse of an incidence matrix. *Journal of the Society for Industrial and Applied Mathematics*, 13(3):827–836, 1965. doi: 10.1137/0113053.

[11] Yuji Ijiri. The beauty of double-entry bookkeeping and its impact on the nature of accounting information. *Economic Notes: Economic Review of Banca Monte dei Paschi di Siena*, 22 (2):265–285, 1993.

[12] International Auditing and Assurance Standards Board. Isa 240: The auditor's responsibilities relating to fraud in an audit of financial statements. IFAC, 2009. Revised June 2024.

[13] International Organization for Standardization. ISO 21378:2019 — audit data collection. Geneva: ISO, 2019.

[14] Michael Ivertowski. Hardware accelerated method for accounting network generation. Internal working paper. On file with the author., 2024.

[15] Michael Ivertowski. *DataSynth – Enterprise Synthetic Data Platform (GitHub repository)*, 2026. URL https://github.com/mivertowski/SyntheticData.

[16] Michael Ivertowski. DataSynth: Reference knowledge graphs for enterprise audit analytics through synthetic data generation with provable statistical properties. SSRN Working Paper, April 2026. URL https://ssrn.com/abstract=6538639. Available at SSRN: https://ssrn.com/abstract=6538639.

[17] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[18] Akim Kotelnikov, Dmitry Baranchuk, Ivan Rubachev, and Artem Babenko. TabDDPM: Modelling tabular data with diffusion models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202, pages 17564–17579, 2023.

[19] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[20] Johannes Lahann, Marvin Scheid, and Peter Fettke. Utilizing machine learning techniques to reveal VAT compliance violations in accounting data. In *2019 IEEE 21st Conference on Business Informatics (CBI)*, 2019.

[21] Pamela J. Liang, Anru Wang, Leman Akoglu, and Christos Faloutsos. Pattern recognition and anomaly detection in bookkeeping data. Carnegie Mellon University, working paper, 2021.

[22] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *IEEE International Conference on Data Mining (ICDM)*, 2008.

[23] Yixin Liu, Kaize Tang, Jiabao Cai, Ao Lu, Zhe Xu, and Shirui Pan. GADBench: Revisiting and benchmarking supervised graph anomaly detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[24] Edgar A. Lopez-Rojas and Stefan Axelsson. BankSim: A bank payments simulator for fraud detection research. In *26th European Modeling and Simulation Symposium (EMSS)*, 2014.

[25] Mark J. Nigrini. Benford's law: Applications for forensic accounting, auditing, and fraud detection. 2012.

[26] Neha Patki, Roy Wedge, and Kalyan Veeramachaneni. The synthetic data vault. In *IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, pages 399–410, 2016.

[27] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *European Semantic Web Conference (ESWC)*, 2018.

[28] Marco Schreyer, Timur Sattarov, Damian Borth, Andreas Dengel, and Bernd Reimer. Detection of anomalies in large-scale accounting data using deep autoencoder networks. In *arXiv preprint arXiv:1709.05254*, 2017.

[29] Marco Schreyer, Timur Sattarov, Bernd Reimer, and Damian Borth. Adversarial learning of deepfakes in accounting. In *NeurIPS Workshop on Robust AI in Financial Services*, 2019.

[30] Aivin V. Solatorio and Olivier Dupriez. REaLTabFormer: Generating realistic relational and tabular data using transformers. *arXiv preprint arXiv:2302.02041*, 2023.

[31] Jiarui Tang, Jian Hua, Ziyi Chen, Mengmei Feng, Xiang Li, and Yulia R. Xu. BOND: Benchmarking unsupervised outlier node detection on static attributed graphs. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[32] Miklos A. Vasarhelyi, Michael G. Alles, and Alexander Kogan. Principles of analytic monitoring for continuous assurance. *Journal of Emerging Technologies in Accounting*, 1(1): 1–21, 2004.

[33] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

[34] Jarrod West and Maumita Bhattacharya. Intelligent financial fraud detection: A comprehensive review. *Computers & Security*, 57:47–66, 2016.

[35] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019.

[36] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. Modeling tabular data using conditional GAN. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

A Released Configuration v1.0

The verbatim YAML below is the configuration that produces the released v1.0 VynFi-Net dataset. It is reproduced in `configs/examples/bench/vynfi_net_v1.yaml` in the DataSynth repository [15] and is the file the released archive is generated from.

Listing 1: Released configuration v1.0 (excerpt). Full file in `configs/examples/bench/vynfi_net_v1.yaml`.

```yaml
global:
  seed: 42
  period_months: 12
```

```yaml
  fiscal_year_start: "2024-01-01"
  industry_profile: manufacturing
  presentation_currency: USD

companies:
  count: 25
  group_structure: enterprise

chart_of_accounts:
  preset: manufacturing
  expand_industry_subaccounts: true

transactions:
  count_per_company_per_month: ~       # set by industry profile

fraud:
  fraud_rate: 0.020
  document_fraud_rate: 0.050
  propagate_to_lines: true

graph_export:
  je_network_csv:
    enabled: true
    methods: [A, B]                    # default; C, D, E disabled
  pytorch_geometric:
    enabled: true
  neo4j:
    enabled: false
  dgl:
    enabled: false

financial_reporting:
  enabled: true                        # produces trial balances
```

B Schema reference

Per-column documentation of `graphs/je_network.csv` appears in Section 6.2 (Table 5). This appendix supplies one worked example of every column for a single edge.

Listing 2: Example row of `graphs/je_network.csv`.

```
edge_id              = 6c4f9a3e-0b7d-5d20-9b9b-7c5e2c2a1f30
document_id          = INV-2024-000123
posting_date         = 2024-07-15
from_account         = 1100             # AR control
to_account           = 4000             # Product revenue
from_line_id         = 7e2c-8a-...
to_line_id           = 9b3d-12-...
amount               = 1250.00
confidence           = 1.00             # Method A
predecessor_edge_id  = b1a2-...         # PO -> GR -> Invoice chain head
```

```
business_process      = O2C
is_fraud              = false
is_anomaly            = false
```

C Baseline hyperparameters

Hyperparameter tables for each baseline appear in the release under `baselines/<method>/hparams.yaml`. All fixed values are reproduced below for reference; the full sweep configurations used for ablations are in the same files under the `sweeps:` key.

*[The hyperparameter tables are pending the v*1.0 *release training run; the YAML files in the repository are the authoritative source.]*